

HACKERRANK 3RD SEMESTER ALGORITHM PORTFOLIO

Student: Mohammed Bilal Ahmed **USN:** R25EF147 **Semester:** 3rd Sem **Language:** Java 8

HackerRank: <https://www.hackerrank.com/profile/mohammedbilalah1>

GitHub: <https://github.com/mohammedbilalahmed45-007>

Portfolio Summary

Problem	Approach	Time	Aux. Space	Evidence supplied
Mini-Max Sum	One-pass scan; track total/min/max	$O(n)$	$O(1)$	Accepted screenshots
Birthday Cake Candles	Track maximum and its frequency	$O(n)$	$O(1)$	Accepted screenshot
Insertion Sort – Part 1	Shift larger values and insert key	$O(n)$	$O(1)$	Accepted + submission screenshots
Binary Search	Iterative halving of sorted interval	$O(\log n)$	$O(1)$	No Binary Search screenshot supplied
Mark and Toys	Sort prices, then greedy purchase	$O(n \log n)$	$O(\log n)^*$	Accepted screenshot

* Java's sorting implementation may use additional runtime memory depending on implementation; auxiliary-space reporting here follows the intended algorithmic model.

Algorithm Notes

- **Mini-Max Sum:** Scan once, maintaining the total and extreme values; exclude max for the minimum four-element sum and exclude min for the maximum.
- **Birthday Cake Candles:** Maintain the tallest value and count how many times it occurs.
- **Insertion Sort – Part 1:** Shift larger elements to the right until the key can be inserted in its correct position.
- **Binary Search:** Compare the target with the middle element and discard half the sorted search interval each iteration.
- **Mark and Toys:** Sort prices and repeatedly buy the cheapest remaining toy while the budget allows.

Before / After Development Evidence

The supplied evidence includes the original VS Code errors caused by multiple files declaring the same public **Solution** class, followed by accepted HackerRank results and a submission page. The portfolio ZIP separates VS Code-friendly uniquely named classes from HackerRank-compatible Solution.java files.

Badge Evidence

The supplied HackerRank profile screenshot shows a **Problem Solving 2-star badge** and a verified **Problem Solving (Basic)** certification.

Reflective Summary

This activity strengthened my ability to translate problem statements into efficient Java algorithms. I learned to reduce unnecessary work by using one-pass scans for Mini-Max Sum and Birthday Cake Candles. Insertion Sort demonstrated how shifting elements solves a local ordering problem while using constant auxiliary space. Binary Search reinforced divide-and-conquer thinking because each comparison removes half of the remaining search interval. Mark and Toys demonstrated how sorting can support a greedy strategy when maximizing purchases under a fixed budget. The tasks also helped me distinguish auxiliary space from total runtime memory and reminded me to consider integer overflow when calculating sums. HackerRank's accepted test results provide practical evidence of correctness, while complexity analysis explains how each solution scales with input size. Finally, resolving the duplicate Java Solution-class issue improved my understanding of how public classes and filenames interact in Java projects.

SCREENSHOT EVIDENCE — ALL SUPPLIED BEFORE/AFTER PHOTOS

All 11 screenshots supplied in this conversation are included below. They are placed according to their role: code/working state, accepted-result evidence, badge/profile evidence, and the VS Code error state.

```

1  import java.io.*;
2  import java.math.*;
3  import java.security.*;
4  import java.text.*;
5  import java.util.*;
6  import java.util.concurrent.*;
7  import java.util.function.*;
8  import java.util.regex.*;
9  import java.util.stream.*;
10 import static java.util.stream.Collectors.joining;
11 import static java.util.stream.Collectors.toList;
12
13 class Result {
14
15     /*
16      * Complete the "miniMaxSum" function below.
17      *
18      * The function accepts INTEGER_ARRAY arr as
19      * parameter.
20      */
21
22     public static void miniMaxSum(List<Integer> arr) {
23         long totalSum = 0;
24         long min = Long.MAX_VALUE;
25         long max = Long.MIN_VALUE;
26
27         for (int num : arr) {

```

01. Before / Mini-Max Sum code

```

39  /
40
41  public class Solution {
42      public static void main(String[] args) throws
43          IOException {
44          BufferedReader bufferedReader = new BufferedReader(
45              (new InputStreamReader(System.in)));
46          BufferedWriter bufferedWriter = new BufferedWriter(
47              (new FileWriter(System.getenv("OUTPUT_PATH"))));
48
49          int candlesCount = Integer.parseInt(
50              (bufferedReader.readLine().trim()));
51
52          List<Integer> candles = Stream.of(bufferedReader.
53              readLine().replaceAll("\\s+", "").split(" "))
54              .map(Integer::parseInt)
55              .collect(toList());
56
57          int result = Result.birthdayCakeCandles(candles);
58
59          bufferedWriter.write(String.valueOf(result));
60          bufferedWriter.newLine();
61
62          bufferedReader.close();
63          bufferedWriter.close();
64      }
65  }

```

02. Birthday Cake Candles code



You have earned 10.00 points!

You are now 10 points away from the 1st star for your problem solving badge.

67%

20/30

Congratulations

You solved this challenge. Would you like to challenge your friends?




Next Challenge


Test case 0


Test case 1


Test case 2


Test case 3


Test case 4


Test case 5

Complex Message

Success

Input (stdin)

```

1 4
3 3 2 1 3

```

Expected Output

```

1 2

```

Download
Download

03. Birthday Cake Candles accepted

```

74 public class Solution {
75     public static void main(String[] args) throws
81         InputStream range(0, t).forEach(itlr -> {
82             try {
83                 int n = Integer.parseInt(bufferedReader.
84                     readLine().trim());
85                 List<Integer> arr = Stream.of
86                     (bufferedReader.readLine().replaceAll("\\s+", "")).split(" ").
87                     .map(Integer::parseInt)
88                     .collect(toList());
89
90                 long result = Result.insertionSort(arr);
91
92                 bufferedWriter.write(String.valueOf
93                     (result));
94                 bufferedWriter.newLine();
95             } catch (IOException ex) {
96                 throw new RuntimeException(ex);
97             }
98         });
99     }
100     bufferedReader.close();
101     bufferedWriter.close();
102 }

```

04. Insertion Sort Part 1 code



You have earned 50.00 points!

You are now 30 points away from the 2nd star for your problem solving badge.

87%

70/100

Congratulations

You solved this challenge. Would you like to challenge your friends?




[Next Challenge](#)


Test case 0


Test case 1


Test case 2


Test case 3


Test case 4


Test case 5

Compiler Message

Success

Input (stdin)

```

1 2
2 5
3 1 1 1 2 2
4 5
5 2 1 1 1 2

```

[Download](#)

05. Insertion Sort Part 1 accepted

```

1 import java.io.*;
2 import java.math.*;
3 import java.security.*;
4 import java.text.*;
5 import java.util.*;
6 import java.util.concurrent.*;
7 import java.util.function.*;
8 import java.util.regex.*;
9 import java.util.stream.*;
10 import static java.util.stream.Collectors.joining;
11 import static java.util.stream.Collectors.toList;
12
13
14 class Result {
15
16     /*
17      * Complete the 'maximumToys' function below.
18      *
19      * The function is expected to return an INTEGER.
20      * The function accepts following parameters:
21      * 1. INTEGER array prices
22      * 2. INTEGER k
23      */
24
25     public static int maximumToys(List<Integer> prices)
26     {
27         Collections.sort(prices);
28     }
29 }

```

Line: 67 Col: 1

06. Mark and Toys code

Congratulations

You solved this challenge. Would you like to challenge your friends?

Next Challenge

Test case 0

Test case 1

Test case 2

Test case 3

Test case 4

Test case 5

Test case 6

Compiler Message

Success

Input (stdin)

1 7 50
2 1 12 5 111 200 1000 10

Download

Expected Output

1 4

Download

07. Mark and Toys accepted

Personal Information

mohammedsalimad5@gmail.com

Add your mobile number

India

My Resume

+ Add Resume

Add your resume here

EEO settings

Orchestrate

How are my skills awarded?

You haven't taken part in any Orchestrate edition yet. [Register for the latest edition](#)

My Badges

Problems Solving
★★

My Certifications

Certificate
Problems Solving (Good)
Verified

08. HackerRank badge/profile evidence

The screenshot shows the IntelliJ IDEA IDE with the 'Problems' tab active. The 'Problems' list on the left contains several error messages. The first error is 'The public type Solution must be defined in its own file' for 'Java\16777541' in 'Ln 3, Col 14'. The second error is 'The type Solution is already defined' for 'Java\16777539' in 'Ln 3, Col 14'. The third error is 'The type Solution is already defined' for 'Java\16777539' in 'Ln 3, Col 14'. The fourth error is 'The type Solution is already defined' for 'Java\16777539' in 'Ln 1, Col 14'. The fifth error is 'The type Solution is already defined' for 'Java\16777539' in 'Ln 1, Col 14'. The sixth error is 'The public type Solution must be defined in its own file' for 'Java\16777541' in 'Ln 3, Col 14'. The 'Solution' class is highlighted in the list. The 'Output' tab is also visible, showing the command 'javac -d . *.java' and the output 'Compilation successful'.

09. Before / VS Code Java class errors

hackerrank.com/challenges/one-week-preparation-kit-mini-max-sum/problem

57

Upload Code as File ☐ Test against custom input

Test case 0 Compiler Message
Success

Test case 1

Test case 2 Input (stdin)
1 1 2 3 4 5

Test case 3 Expected Output
1 10 14

Test case 4

Test case 5

Test case 6

10. Mini-Max Sum accepted

hackerrank.com/challenges/insertion-sort/submissions/code/484096176

Language: Java 8 [Open in editor](#)

Test case 0 Success

Test case 1

Test case 2 Input (stdin)
1 2

Test case 3 2 5

Test case 4 3 1 1 1 2 2

Test case 5 4 5

Test case 6 5 2 1 3 1 2

Test case 7 Expected Output
1 0

Test case 8 2 4

11. Insertion Sort submission accepted